Proving Data Structures Correct: Balanced Search Trees

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

Insertion: The Idea

©Mohammad Abdulaziz

Insertion: The Idea

`Leaf` $\rightsquigarrow$ `Node2`

©Mohammad Abdulaziz

Insertion: The Idea

```
Leaf   ⇝   Node2
Node2  ⇝   Node3
```

©Mohammad Abdulaziz

Insertion: The Idea

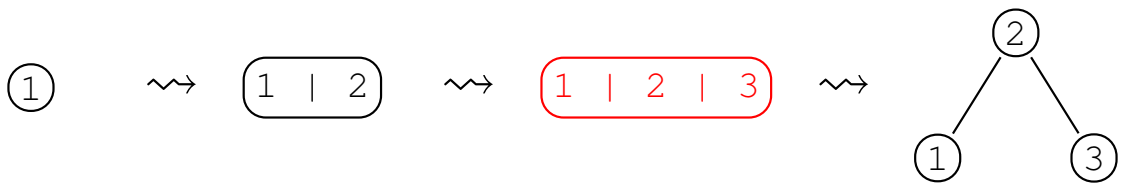
`Leaf`	$\rightsquigarrow$	`Node2`
`Node2`	$\rightsquigarrow$	`Node3`
`Node3`	$\rightsquigarrow$	overflow, pass 1 element back up

©Mohammad Abdulaziz

Insertion: The Idea

`Leaf`	$\rightsquigarrow$	`Node2`
`Node2`	$\rightsquigarrow$	`Node3`
`Node3`	$\rightsquigarrow$	overflow, pass 1 element back up

Inserting 1, 2, 3:

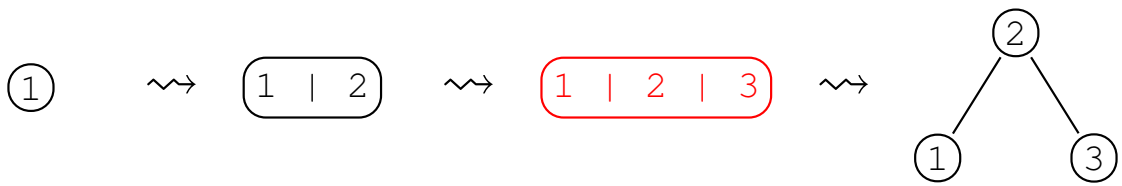


©Mohammad Abdulaziz

Insertion: The Idea

`Leaf`	$\rightsquigarrow$	`Node2`
`Node2`	$\rightsquigarrow$	`Node3`
`Node3`	$\rightsquigarrow$	overflow, pass 1 element back up

Inserting 1, 2, 3:



The tree grows at the root: all leaves stay at the same level

©Mohammad Abdulaziz

Insertion: Two Possible Results

`ins x t` either

©Mohammad Abdulaziz

Insertion: Two Possible Results

`ins x t` either

- accommodates `x` without increasing the height: $\texttt{Eq}_i\ \texttt{t'}$

Insertion: Two Possible Results

`ins x t` either

- accommodates `x` without increasing the height: $\texttt{Eq}_i$ `t'`
- overflows: `Of l a r`

©Mohammad Abdulaziz

Insertion: Two Possible Results

`ins x t` either

- accommodates `x` without increasing the height: `Eq`$_i$ `t'`
- overflows: `Of l a r`

```
datatype 'a up_i = Eq_i ('a tree23) | Of ('a tree23) 'a ('a tree23)
```

Insertion: Two Possible Results

`ins x t` either

- accommodates `x` without increasing the height: `Eq`$_i$ `t'`
- overflows: `Of l a r`

```
datatype 'a up_i = Eq_i ('a tree23) | Of ('a tree23) 'a ('a tree23)

tree_i :: 'a up_i ⇒ 'a tree23
tree_i (Eq_i t) = t
tree_i (Of l a r) = ⟨l, a, r⟩
```

Insertion: Two Possible Results

`ins x t` either

- accommodates `x` without increasing the height: `Eq`$_i$ `t'`
- overflows: `Of l a r`

```
datatype 'a up_i = Eq_i ('a tree23) | Of ('a tree23) 'a ('a tree23)

tree_i :: 'a up_i ⇒ 'a tree23
tree_i (Eq_i t) = t
tree_i (Of l a r) = ⟨l, a, r⟩

insert x t = tree_i (ins x t)
ins :: 'a ⇒ 'a tree23 ⇒ 'a up_i
```

Insertion: `Leaf` and `Node2`

```
ins x Leaf = Of Leaf x Leaf
```

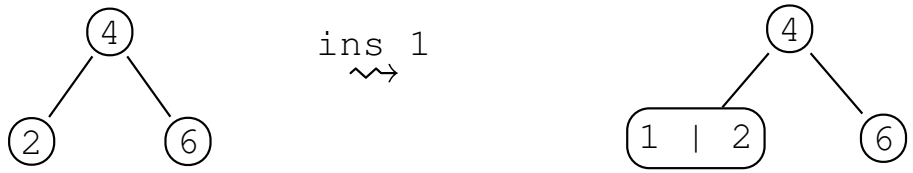
©Mohammad Abdulaziz

Insertion: `Leaf` and `Node2`

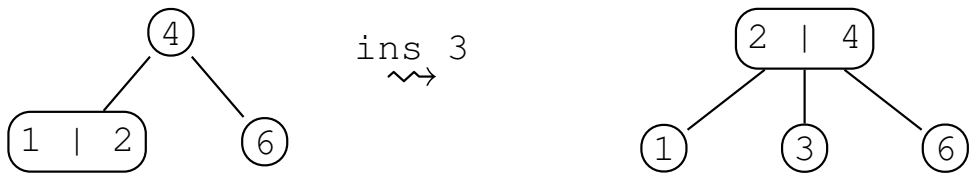
```
ins x Leaf = Of Leaf x Leaf

ins x ⟨l, a, r⟩ =
  (if x < a then (case ins x l of
                    Eqi l' ⇒ Eqi ⟨l', a, r⟩
                  | Of l1 b l2 ⇒ Eqi ⟨l1, b, l2, a, r⟩) else
   if x > a then (case ins x r of
                    Eqi r' ⇒ Eqi ⟨l, a, r'⟩
                  | Of r1 b r2 ⇒ Eqi ⟨l, a, r1, b, r2⟩)
   else Eqi ⟨l, a, r⟩)
```

Insertion into `Node2`: Examples



`Leaf` overflows; the 2-node `2` absorbs `1` and becomes a 3-node: $\texttt{Eq}_i$



`1 | 2` overflows: `Of` ⟨`1`⟩ `2` ⟨`3`⟩; the 2-node `4` absorbs `2`: $\texttt{Eq}_i$

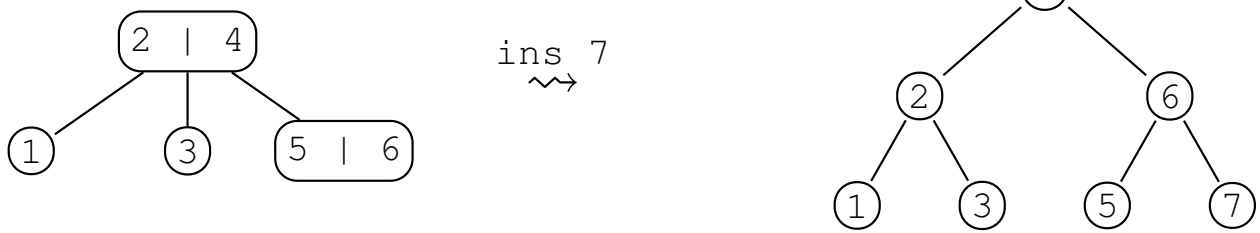
©Mohammad Abdulaziz

Insertion: Node3

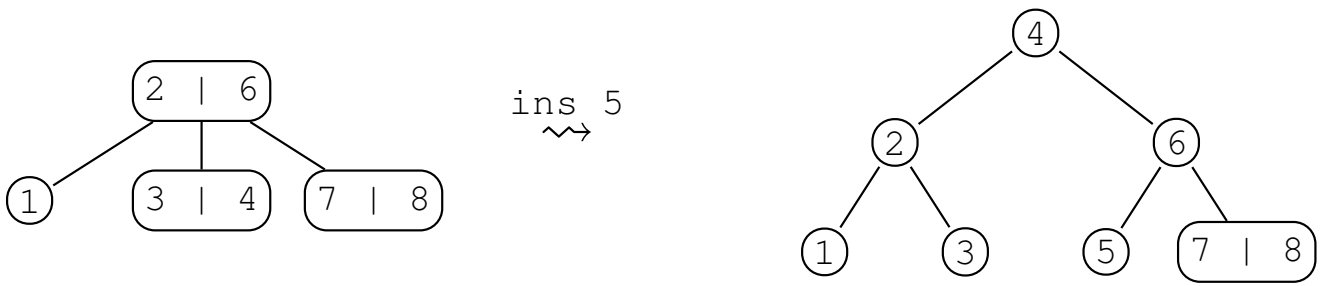
```
ins x ⟨l, a, m, b, r⟩ =
  (if x < a then (case ins x l of
                    Eq_i l' ⇒ Eq_i ⟨l', a, m, b, r⟩
                  | Of l1 c l2 ⇒ Of ⟨l1, c, l2⟩ a ⟨m, b, r⟩) else
   if x = a then Eq_i ⟨l, a, m, b, r⟩ else
   if x < b then (case ins x m of
                    Eq_i m' ⇒ Eq_i ⟨l, a, m', b, r⟩
                  | Of m1 c m2 ⇒ Of ⟨l, a, m1⟩ c ⟨m2, b, r⟩) else
   if x = b then Eq_i ⟨l, a, m, b, r⟩
   else (case ins x r of
           Eq_i r' ⇒ Eq_i ⟨l, a, m, b, r'⟩
         | Of r1 c r2 ⇒ Of ⟨l, a, m⟩ b ⟨r1, c, r2⟩))
```

©Mohammad Abdulaziz

Insertion into `Node3`: Examples



`5 | 6` overflows, then `2 | 4` overflows: `insert` makes `4` the new root, the height grows



middle case: `3 | 4` overflows: `Of ⟨3⟩ 4 ⟨5⟩`; `4` moves up, `2 | 6` splits around it

Insertion Preserves `complete`?

Goal: `complete t` $\Longrightarrow$ `complete (insert x t)`

Proof attempt by induction on `t`.
Case `t` $=\langle$`l, a, r`$\rangle$ with `x < a`:

- IH: `complete l` $\Longrightarrow$ `complete (insert x l)`
- Assumption: `complete` $\langle$`l, a, r`$\rangle$, i.e. $h(l) = h(r)$, `complete l`, `complete r`
- `ins x l` $=$ `Eq`$_i$ `l'`: `insert x t` $=\langle$`l', a, r`$\rangle$
  IH: `complete l'` ✓, `complete r` ✓, but $h(l') = h(r)$? unknown
- `ins x l` $=$ `Of l1 b l2`: `insert x t` $=\langle$`l1, b, l2, a, r`$\rangle$
  IH: `complete` $\langle$`l1, b, l2`$\rangle$ ✓, but $h(l1) = h(r)$? unknown

$\Rightarrow$ we also need to know the height of the result of `ins`

©Mohammad Abdulaziz

Generalising the Statement

Lemma

`complete t` $\Longrightarrow$
`  complete (tree`$_i$ `(ins x t)) ∧ h`$_i$`(ins x t) = h(t)`

where `h`$_i$ `(Eq`$_i$ `t) = h(t), h`$_i$ `(Of l a r) = h(l)`

Proof by induction on `t`. Case `t = ⟨l, a, r⟩` with `x < a`:

- IH: `complete l` $\Longrightarrow$
  `complete (tree`$_i$ `(ins x l)) ∧ h`$_i$`(ins x l) = h(l)`
- Assumption: $h(l) = h(r) = k$, `complete l`, `complete r`
- `ins x l = Eq`$_i$ `l'`: IH: `complete l'`, $h(l') = k$
  `ins x t = Eq`$_i$ `⟨l', a, r⟩`: complete, `h`$_i = k + 1 = h(t)$ ✓
- `ins x l = Of l1 b l2`: IH: `complete ⟨l1, b, l2⟩`, $h(l1) = k$, so $h(l2) = k$
  `ins x t = Eq`$_i$ `⟨l1, b, l2, a, r⟩`: complete, `h`$_i = k + 1 = h(t)$ ✓

©Mohammad Abdulaziz

Insertion Preserves `complete`

Lemma

`complete t` $\Longrightarrow$
`  complete (tree`$_i$ `(ins x t)) ∧ h`$_i$`(ins x t) = h(t)`

Proof by induction on `t`; the other cases are similar.
In Isabelle: `by (induction t) (auto split:  up`$_i$`.splits)`

Insertion Preserves `complete`

Lemma

`complete t` $\Longrightarrow$
`  complete (tree`$_i$ `(ins x t)) ∧ h`$_i$`(ins x t) = h(t)`

Proof by induction on `t`; the other cases are similar.
In Isabelle: `by (induction t) (auto split:  up`$_i$`.splits)`

Corollary

`complete t` $\Longrightarrow$ `complete (insert x t)`

Beyond 2-3 Trees

©Mohammad Abdulaziz

Beyond 2-3 Trees

Deletion: `Node2` $\rightsquigarrow$ underflow, merge with siblings on the way up

©Mohammad Abdulaziz

Beyond 2-3 Trees

Deletion: `Node2` $\rightsquigarrow$ underflow, merge with siblings on the way up

datatype `'a tree234 =`
`  Leaf | Node2 ... | Node3 ... | Node4 ...`

Like 2-3 trees, but with more cases

©Mohammad Abdulaziz

Beyond 2-3 Trees

Deletion: `Node2` $\rightsquigarrow$ underflow, merge with siblings on the way up

datatype `'a tree234 =`
`  Leaf | Node2 ... | Node3 ... | Node4 ...`

Like 2-3 trees, but with more cases

The general case:

B-trees and (a, b)-trees